DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Banking Database Management System

Complex Computing Problem & Activity-CCP

Project Title	Banking Management System
Course Code	CT-261
Course Name	Database Management System

Group Members		
S1	HUZAIFA JAWED	| Roll No: CT-24077
S2	SAIM UZ ZAMAN	| Roll No: CT-24078
S3	FOZAN AHMAD KHAN	| Roll No: CT-24096
S4	M. FOUZAN ABDUL AZIZ	| Roll No: CT-24090

1. Organization

1.1 Problem Statement

Traditional banking institutions manage customer accounts, transactions, loans, and card services through fragmented, siloed systems — often built on flat-file databases or legacy spreadsheet-based records. These systems suffer from critical data management failures that directly impact operational efficiency, regulatory compliance, and customer satisfaction.

The core challenge is the absence of a unified relational database capable of enforcing referential integrity, supporting multi-entity relationships (such as joint accounts with multiple owners), and providing real-time analytical reporting across branches, customers, and financial products.

1.2 Business Requirements

The proposed Banking DBMS must fulfil the following functional requirements:

- **Manage multiple account types: Savings (with minimum balance and withdrawal limits), Current (with overdraft and cheque book), and Fixed Deposits (with maturity dates and penalty clauses)**
- Support two classes of customers: Individual (tracked by NIC, nationality, income level) and Corporate (tracked by company registration, industry type)
- Enable joint account ownership with configurable ownership shares stored in a junction relation
- Record and classify transactions as Deposits, Withdrawals, or Transfers (including inter-bank transfers via IBAN to External Customers)
- Issue and lifecycle-manage Debit and Credit cards linked to customer accounts, tracking CVV, expiry, issue date, and card status
- Handle the full loan lifecycle: application, approval, EMI schedule, remaining balance, overdue fines, and per-payment audit trail via LoanPayment
- Classify employees into three specialized roles: Manager, Teller, and Loan Officer — each with role-specific attributes while sharing a base Employee relation
- Assign all entities (accounts, employees, loans) to specific branches with full branch metadata

1.3 Limitations of the Current (Pre-DBMS) System

- Customer data duplicated across branch registers, causing update anomalies
- No audit trail linking a transaction to the employee who processed it
- Joint account ownership can’t be enforced or tracked only single-owner accounts supported
- Card lifecycle (expiry,blocking,CVV) managed in separate spreadsheets,not linked to accounts
- No support for multi-currency or external bank transfers with IBAN validation

2. Conceptual Model

2.1 Entity-Relationship Diagram (ERD) — Overview

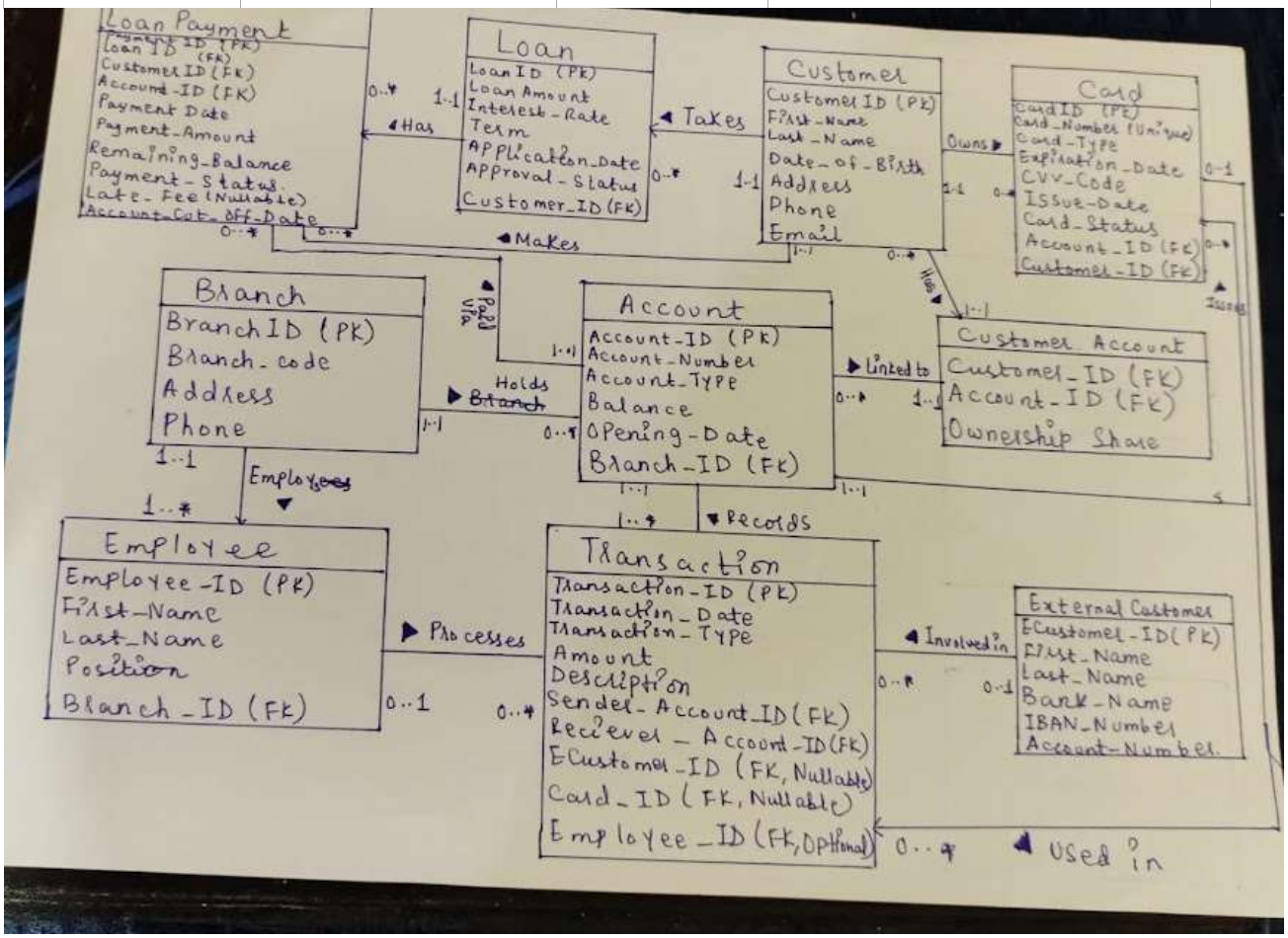
The conceptual model identifies all entities, their attributes, and the relationships between them. The ERD was hand-drawn and reviewed in detail. The following entities and their primary attributes form the backbone of the system:

Entity	Key Attributes
Account	`Account_ID (PK), Account_Number, Account_Type, Balance, Opening_Date, Branch_ID (FK)`
Customer	`Customer_ID (PK), First_Name, Last_Name, Date_of_Birth, Address, Phone, Email`
Branch	`Branch_ID (PK), Branch_Code, Branch_Name, Address, Phone`
Employee	`Employee_ID (PK), First_Name, Last_Name, Position, Branch_ID (FK)`
Loan	`Loan_ID (PK), Loan_Amount, Interest_Rate, Term, Remaining_Balance, Payment_Status, Customer_ID (FK), Branch_ID (FK)`
Card	`Card_ID (PK), Card_Number, Card_Type, Expiration_Date, CVV_Code, Issue_Date, Card_Status, Account_ID (FK), Customer_ID (FK)`
Transaction	`Transaction_ID (PK), Transaction_Date, Transaction_Type, Amount, Description, Sender_Account_ID (FK), Receiver_Account_ID (FK), Card_ID (FK)`
CustomerAccount	`Customer_ID (FK) + Account_ID (FK) → Composite PK, Ownership_Share`
ExternalCustomer	`ECustomer_ID (PK), First_Name, Last_Name, Bank_Name, IBAN_Number, Account_Number`
LoanPayment	`LoanPayment_ID (PK), Loan_ID (FK), Payment_Date, Amount_Paid`

2.2 Relationships and Cardinalities

Relationship	Entities Involved	Cardinality	Description
Issues	Customer — Card	1..* to 0..*	A customer may own one or more cards; a card is issued to one customer
Linked To	Customer — Account (via Customer_Account)	M:N	Multiple customers can share an account; a customer can hold many accounts
Holds	Branch — Account	1 to 0..*	A branch holds many accounts; each account belongs to exactly one branch
Employs	Branch — Employee	1 to 1..*	A branch has at least one employee; each employee works at one branch

Relationship	Entities Involved	Cardinality	Description
Makes	Customer — Loan	1 to 0..*	A customer may take multiple loans
Has	Loan — Loan_Payment	1 to 0..*	Each loan has zero or more payment records
Records	Account — Transaction	1 to 0..*	An account records many transactions (as sender or receiver)
Processes	Employee — Transaction	1 to 0..*	An employee optionally processes a transaction
Used In	Card — Transaction	1 to 0..*	A card may be used in transactions
Involved In	External_Customer — Transaction	1 to 0..*	External customer may appear in transactions
Paid Via	Loan_Payment — Account	1 to 1	Payment is made via an account



3. Enhanced Entity-Relationship Model (EERD)

3.1 Generalizations and Specializations

The EERD extends the ERD with inheritance hierarchies using generalization and specialization to reduce redundancy and capture entity-specific attributes cleanly.

Superclass	Participation	Subclasses	Design Rationale
Account (Superclass)	**Mandatory-Or (Total)**	SavingAccount, CurrentAccount, FixedDeposit	Specialization by account product type
Customer (Superclass)	**Mandatory-Or (Total)**	IndividualCustomer, CorporateCustomer	Specialization by legal entity type
Employee (Superclass)	**Optional-Or (Partial)**	Manager, Teller, LoanOfficer	Specialization by job role — an employee may have no sub-role
Transaction (Superclass)	**Mandatory-Or (Total)**	Deposit, Withdrawal, Transfer	Specialization by transaction operation type

3.2 Aggregation

The relationship "Used In" between Transaction and ExternalCustomer captures aggregation: a complete transaction (as a relationship between Account and Branch) is treated as a higher-level entity that interacts with the ExternalCustomer entity. This models inter-bank wire transfers where the entire internal transaction context is required to form the external relationship.

3.3 Subclass Attribute Details

Account Specializations

- SavingAccount: Min_Balance (DECIMAL), Payment_Rate (%), Withdrawal_Limit (INT)
- CurrentAccount: Overdraft_Limit (DECIMAL), Funds_Transfer (BOOLEAN), Cheque_Book (BOOLEAN)
- FixedDeposit: Maturity_Date (DATE), Fixed_Rate (%), Penalty_Charge (DECIMAL)

Customer Specializations

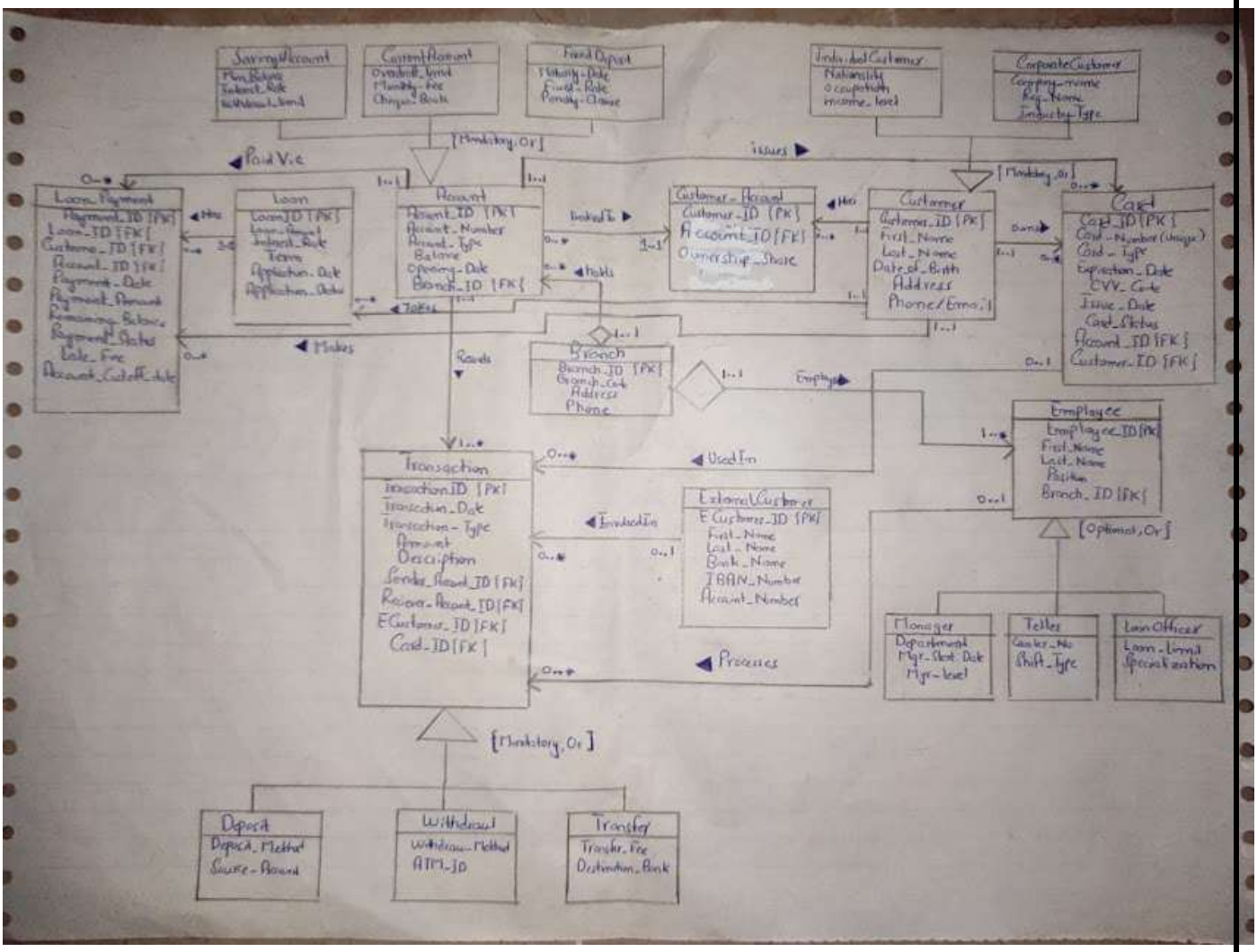
- IndividualCustomer: Nationality (VARCHAR), NIC (VARCHAR UNIQUE), Income_Level (DECIMAL)
- CorporateCustomer: Company_Name (VARCHAR), Reg_Number (VARCHAR UNIQUE), Industry_Type (VARCHAR)

Employee Specializations

- Manager: Department (VARCHAR), Mgt_Start_Date (DATE), Mgt_Level (VARCHAR)
- Teller: Counter_No (INT), Shift_Type (ENUM: Morning/Evening/Night)
- LoanOfficer: Loan_Limit (DECIMAL), Specialization (VARCHAR)

Transaction Specializations

- Deposit: Deposit_Method (VARCHAR), Source_Amount (DECIMAL)
- Withdrawal: Withdraw_Method (VARCHAR), ATM_ID (VARCHAR)
- Transfer: Transfer_Fee (DECIMAL), Destination_Bank (VARCHAR)



4. Functional Dependencies

The following functional dependencies were identified from the business rules and entity attributes. These form the mathematical basis for the normalization process that follows.

FD 4.1 — Branch

Branch_ID	→	Branch_Code, Branch_Name, Address, Phone

FD 4.2 — Customer

Customer_ID	→	First_Name, Last_Name, Date_of_Birth, Address, Phone, Email

FD 4.3 — IndividualCustomer

Customer_ID	→	Nationality, NIC, Income_Level

FD 4.4 — CorporateCustomer

Customer_ID	→	Company_Name, Reg_Number, Industry_Type

FD 4.5 — Account

Account_ID	→	Account_Number, Account_Type, Balance, Opening_Date, Branch_ID

FD 4.6 — CustomerAccount (Junction)

Customer_ID, Account_ID	→	Ownership_Share

FD 4.7 — Employee

Employee_ID	→	First_Name, Last_Name, Position, Branch_ID

FD 4.8 — Manager

Employee_ID	→	Department, Mgt_Start_Date, Mgt_Level

FD 4.9 — Teller

Employee_ID	→	Counter_No, Shift_Type

FD 4.10 — LoanOfficer

Employee_ID	→	Loan_Limit, Specialization

FD 4.11 — Loan

Loan_ID	→	Loan_Number, Loan_Amount, Interest_Rate, Term, Application_Date, Approval_Date, Payment_Amount, Remaining_Balance, Payment_Status, Due_Date, Due_Fine, Customer_ID, Branch_ID

FD 4.12 — LoanPayment

LoanPayment_ID	→	Loan_ID, Payment_Date, Amount_Paid

FD 4.13 — Card

Card_ID	→	Card_Number, Card_Type, Expiration_Date, CVV_Code, Issue_Date, Card_Status, Account_ID, Customer_ID

FD 4.14 — Transaction

Transaction_ID	→	Transaction_Date, Transaction_Type, Amount, Description, Sender_Account_ID, Receiver_Account_ID, ECustomer_ID, Card_ID

FD 4.15 — ExternalCustomer

ECustomer_ID	→	First_Name, Last_Name, Bank_Name, IBAN_Number, Account_Number

FD 4.16 — SavingAccount

Account_ID	→	Min_Balance, Payment_Rate, Withdrawal_Limit

FD 4.17 — FixedDeposit

Account_ID	→	Maturity_Date, Fixed_Rate, Penalty_Charge

FD 4.18 — CurrentAccount

Account_ID	→	Overdraft_Limit, Funds_Transfer, Cheque_Book

FD 4.19 — Deposit

Transaction_ID	→	Deposit_Method, Source_Amount

FD 4.20 — Withdrawal

Transaction_ID	→	Withdraw_Method, ATM_ID

FD 4.21 — Transfer

Transaction_ID	→	Transfer_Fee, Destination_Bank

5. Normalization Process (1NF to BCNF)

Five representative tables are normalized step-by-step from an Unnormalized Form (UNF) through 1NF, 2NF, 3NF, and BCNF. Each stage shows the violation found and the fix applied.

TABLE 1 — Customer

UNF Unnormalized Form

- Relation: Customer(Customer_ID, Customer_Name, Phone1, Phone2, Address, Email, NIC, Nationality, Income_Level, Company_Name, Reg_Number, Industry_Type)
- Problem 1: Phone1 and Phone2 are repeating groups — multiple phone values violate atomicity.
- Problem 2: NIC/Nationality/Income_Level are individual-only; Company_Name/Reg_Number/Industry_Type are corporate-only — mixed entity types in one flat table.

1NF First Normal Form — Remove Repeating Groups

- Fix: Merge Phone1/Phone2 into one atomic Phone column. Assign Customer_ID as primary key.
- Result: Customer(Customer_ID, First_Name, Last_Name, Phone, Address, Email, NIC, Nationality, Income_Level, Company_Name, Reg_Number, Industry_Type)
- ✓ Atomic values. ✓ Primary key defined. In 1NF.

2NF Second Normal Form — No Partial Dependencies

- Primary key is Customer_ID (single attribute) → partial dependency cannot exist by definition.
- ✓ All non-key attributes fully depend on Customer_ID. Already in 2NF.

3NF Third Normal Form — Remove Transitive Dependencies

- Violation: NIC → Nationality (NIC determines Nationality — transitive chain through Customer_ID). Also, individual/corporate attributes do not apply to all customers.
- Fix: Split into: Customer(Customer_ID, First_Name, Last_Name, Phone, Address, Email) | IndividualCustomer(Customer_ID, NIC, Nationality, Income_Level) | CorporateCustomer(Customer_ID, Company_Name, Reg_Number, Industry_Type)
- ✓ No transitive dependencies remain. All three relations in 3NF.

BCNF Boyce-Codd Normal Form

- Customer: Customer_ID → all attributes. Customer_ID is the sole superkey. ✓
- IndividualCustomer: Customer_ID → {NIC, Nationality, Income_Level}. Customer_ID is PK and superkey. ✓
- CorporateCustomer: Customer_ID → {Company_Name, Reg_Number, Industry_Type}. Customer_ID is PK and superkey. ✓
- FINAL: Customer → 3 tables, all in BCNF.

TABLE 2 — Loan

UNF Unnormalized Form

- Relation: Loan(Loan_ID, Loan_Amount, Interest_Rate, Term, Application_Date, Customer_Name, Customer_Phone, Branch_Name, Branch_Address, Payment1_Date, Payment1_Amount, Payment2_Date, Payment2_Amount)
- Problem 1: Payment1 and Payment2 columns are repeating groups — a loan can have many payments over its term.
- Problem 2: Customer_Name, Customer_Phone, Branch_Name, Branch_Address are full attribute copies from other entities — data redundancy.

1NF First Normal Form

- Fix: Remove Payment1/Payment2 repeating group. Create: LoanPayment(LoanPayment_ID, Loan_ID, Payment_Date, Amount_Paid).
- Fix: Replace customer/branch full copies with foreign keys: Customer_ID, Branch_ID.
- Result: Loan(Loan_ID, Loan_Number, Loan_Amount, Interest_Rate, Term, Application_Date, Approval_Date, Remaining_Balance, Payment_Status, Due_Date, Due_Fine, Customer_ID, Branch_ID)
- ✓ Atomic. ✓ Primary keys defined. In 1NF.

2NF Second Normal Form

- Loan PK = Loan_ID (single). LoanPayment PK = LoanPayment_ID (single). No composite keys → partial dependency impossible.
- ✓ Both relations already in 2NF.

3NF Third Normal Form

- Check Loan: Loan_ID → Branch_ID → Branch_Address — transitive chain broken because Branch_Address is NOT stored in Loan (only Branch_ID FK). ✓
- Check LoanPayment: No non-key attribute depends on another non-key attribute. ✓

- ✓ No transitive dependencies. Both relations in 3NF.

BCNF Boyce-Codd Normal Form

- Loan: Loan_ID and Loan_Number are both candidate keys (superkeys). Every determinant is a superkey. ✓
- LoanPayment: Only FD is LoanPayment_ID → {Loan_ID, Payment_Date, Amount_Paid}. LoanPayment_ID is the sole superkey. ✓
- FINAL: Loan + LoanPayment both in BCNF.

TABLE 3 — Account (with CustomerAccount Junction)

UNF Unnormalized Form

- Relation: Account(Account_ID, Account_Number, Account_Type, Balance, Branch_Name, Branch_Phone, Owner1_ID, Owner1_Name, Owner1_Share, Owner2_ID, Owner2_Name, Owner2_Share, Min_Balance, Overdraft_Limit, Maturity_Date)
- Problem 1: Owner1/Owner2 are repeating groups — joint accounts can have multiple owners.
- Problem 2: Branch_Name, Branch_Phone are full copies — data redundancy.
- Problem 3: Min_Balance (Saving), Overdraft_Limit (Current), Maturity_Date (Fixed) — type-specific columns mixed for all account types.

1NF First Normal Form

- Fix: Remove Owner1/Owner2 repeating groups → create CustomerAccount(Customer_ID, Account_ID, Ownership_Share) with composite PK.
- Fix: Replace Branch_Name/Phone with Branch_ID (FK). Remove type-specific columns from Account.
- Result: Account(Account_ID, Account_Number, Account_Type, Balance, Opening_Date, Branch_ID)
- ✓ Atomic. ✓ Primary keys defined. In 1NF.

2NF Second Normal Form

- Account PK = Account_ID (single) → no partial dependency.
- CustomerAccount PK = (Customer_ID, Account_ID). Ownership_Share depends on BOTH — the share is specific to a customer-account pair. No partial dependency. ✓
- ✓ Both relations in 2NF.

3NF Third Normal Form

- Account: Branch details not stored (only Branch_ID FK) → no transitive chain. ✓
- Type-specific attributes separated: SavingAccount(Account_ID, Min_Balance, Payment_Rate, Withdrawal_Limit) | CurrentAccount(Account_ID, Overdraft_Limit, Funds_Transfer, Cheque_Book) | FixedDeposit(Account_ID, Maturity_Date, Fixed_Rate, Penalty_Charge). ✓
- ✓ No transitive dependencies. All relations in 3NF.

BCNF Boyce-Codd Normal Form

- Account: Account_ID and Account_Number are both superkeys. Every determinant is a superkey. ✓
- CustomerAccount: {Customer_ID, Account_ID} → Ownership_Share. LHS is full composite PK (superkey). ✓
- SavingAccount / CurrentAccount / FixedDeposit: Each has Account_ID as sole determinant and PK. ✓
- FINAL: Account hierarchy (5 tables) all in BCNF.

TABLE 4 — Employee

UNF Unnormalized Form

- Relation: Employee(Employee_ID, First_Name, Last_Name, Position, Branch_ID, Branch_Name, Branch_Address, Department, Mgt_Start_Date, Mgt_Level, Counter_No, Shift_Type, Loan_Limit, Specialization)
- Problem 1: Branch_Name and Branch_Address are full branch details duplicated here — redundant storage.
- Problem 2: Department/Mgt_Level (Manager only), Counter_No/Shift_Type (Teller only), Loan_Limit/Specialization (LoanOfficer only) — all mixed, causing massive NULL values per row.

1NF First Normal Form

- Fix: Remove Branch_Name, Branch_Address — keep only Branch_ID (FK).
- Fix: Remove all role-specific columns from Employee.
- Result: Employee(Employee_ID, First_Name, Last_Name, Position, Branch_ID)
- ✓ Atomic. ✓ Primary key defined. In 1NF.

2NF Second Normal Form

- PK = Employee_ID (single attribute). Partial dependency impossible.
- ✓ All attributes fully depend on Employee_ID. In 2NF.

3NF Third Normal Form

- Employee_ID → Branch_ID → {Branch_Name, ...} — transitive chain already eliminated (Branch details not in Employee). ✓
- Role-specific subtype tables: Manager(Employee_ID, Department, Mgt_Start_Date, Mgt_Level) | Teller(Employee_ID, Counter_No, Shift_Type) | LoanOfficer(Employee_ID, Loan_Limit, Specialization). ✓
- ✓ No transitive dependencies. All relations in 3NF.

BCNF Boyce-Codd Normal Form

- Employee: Employee_ID → all attributes. Employee_ID is sole superkey. ✓
- Manager / Teller / LoanOfficer: Each has Employee_ID as PK and sole determinant. ✓
- FINAL: Employee hierarchy (4 tables) all in BCNF.

TABLE 5 — Transaction

UNF Unnormalized Form

- Relation: Transaction(Transaction_ID, Transaction_Date, Transaction_Type, Amount, Sender_Account_Number, Sender_Balance, Receiver_Account_Number, Receiver_Balance, Card_Number, Card_Type, Card_Expiry, Deposit_Method, Withdraw_Method, ATM_ID, Transfer_Fee, Destination_Bank)
- Problem 1: Sender_Balance and Receiver_Balance are derived values — storing them causes update anomalies whenever account balance changes.
- Problem 2: Card_Number, Card_Type, Card_Expiry are full Card attribute copies instead of an FK reference.
- Problem 3: Deposit_Method, Withdraw_Method, ATM_ID, Transfer_Fee, Destination_Bank — type-specific columns for all transaction types, causing NULLs.

1NF First Normal Form

- Fix: Remove Sender_Balance/Receiver_Balance (derived — fetched from Account at query time).
- Fix: Replace Card_Number/Type/Expiry with Card_ID (FK). Replace Account_Numbers with Account_ID FKs.
- Fix: Remove type-specific columns from Transaction.

- Result: Transaction(Transaction_ID, Transaction_Date, Transaction_Type, Amount, Description, Sender_Account_ID, Receiver_Account_ID, ECustomer_ID, Card_ID)
- ✓ Atomic. ✓ Primary key defined. In 1NF.

2NF Second Normal Form

- PK = Transaction_ID (single). Partial dependency impossible.
- ✓ All attributes depend fully on Transaction_ID. In 2NF.

3NF Third Normal Form

- Transaction_ID → Card_ID → {Card_Number, Card_Type} — transitive chain broken (Card details not stored in Transaction). ✓
- Type-specific subtypes: Deposit(Transaction_ID, Deposit_Method, Source_Amount) | Withdrawal(Transaction_ID, Withdraw_Method, ATM_ID) | Transfer(Transaction_ID, Transfer_Fee, Destination_Bank). ✓
- ✓ No transitive dependencies remain. All relations in 3NF.

BCNF Boyce-Codd Normal Form

- Transaction: Transaction_ID is the sole determinant and superkey. ✓
- Deposit / Withdrawal / Transfer: Each has Transaction_ID as PK and sole determinant. ✓
- FINAL: Transaction hierarchy (4 tables) all in BCNF. Schema is free of all anomalies.

6.1 Final Normalized Schema Summary

Table Name	Primary Key	Foreign Keys	Normal Form Achieved
Branch	Branch_ID	—	**BCNF**
Customer	Customer_ID	—	**BCNF**
IndividualCustomer	Customer_ID	Customer_ID → Customer	**BCNF**
CorporateCustomer	Customer_ID	Customer_ID → Customer	**BCNF**
Account	Account_ID	Branch_ID → Branch	**BCNF**
SavingAccount	Account_ID	Account_ID → Account	**BCNF**
CurrentAccount	Account_ID	Account_ID → Account	**BCNF**
FixedDeposit	Account_ID	Account_ID → Account	**BCNF**
CustomerAccount	(Customer_ID, Account_ID)	Both FKs	**BCNF**

Table Name	Primary Key	Foreign Keys	Normal Form Achieved
Employee	Employee_ID	Branch_ID → Branch	**BCNF**
Manager	Employee_ID	Employee_ID → Employee	**BCNF**
Teller	Employee_ID	Employee_ID → Employee	**BCNF**
LoanOfficer	Employee_ID	Employee_ID → Employee	**BCNF**
ExternalCustomer	ECustomer_ID	—	**BCNF**
Loan	Loan_ID	Customer_ID, Branch_ID	**BCNF**
LoanPayment	LoanPayment_ID	Loan_ID → Loan	**BCNF**
Card	Card_ID	Account_ID, Customer_ID	**BCNF**
Transaction	Transaction_ID	Sender/Receiver_Account_ID, ECustomer_ID, Card_ID	**BCNF**
Deposit	Transaction_ID	Transaction_ID → Transaction	**BCNF**
Withdrawal	Transaction_ID	Transaction_ID → Transaction	**BCNF**
Transfer	Transaction_ID	Transaction_ID → Transaction	**BCNF**

7. Physical Design — DDL Schema

```
CREATE DATABASE IF NOT EXISTS BankingManagementSystem;
USE BankingManagementSystem;

-- ----------------------------------------------------------
-- TABLE: Branch
-- ----------------------------------------------------------
CREATE TABLE Branch (
    Branch_ID      INT           NOT NULL AUTO_INCREMENT,
    Branch_Code    VARCHAR(20)   NOT NULL UNIQUE,
    Address        VARCHAR(255)  NOT NULL,
    Phone          VARCHAR(20),
    PRIMARY KEY (Branch_ID)
);

-- ----------------------------------------------------------
-- TABLE: Customer
-- ----------------------------------------------------------
CREATE TABLE Customer (
    Customer_ID    INT           NOT NULL AUTO_INCREMENT,
    First_Name     VARCHAR(100)  NOT NULL,
    Last_Name      VARCHAR(100)  NOT NULL,
    Nationality    VARCHAR(100),
    Occupation     VARCHAR(100),
    Income_Level   VARCHAR(50),
    Date_of_Birth  DATE,
    Address        VARCHAR(255),
    Phone          VARCHAR(20),
    Email          VARCHAR(150),
    PRIMARY KEY (Customer_ID)
);

-- ----------------------------------------------------------
-- TABLE: Individual_Customer (EER Subtype of Customer)
-- ----------------------------------------------------------
CREATE TABLE Individual_Customer (
    Customer_ID    INT           NOT NULL,
    Nationality    VARCHAR(100),
    Occupation     VARCHAR(100),
    Income_Level   VARCHAR(50),
    PRIMARY KEY (Customer_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
        ON DELETE CASCADE
);





-- ----------------------------------------------------------
```

```
-- TABLE: Corporate_Customer (EER Subtype of Customer)
-- ----------------------------------------------------------
CREATE TABLE Corporate_Customer (
    Customer_ID    INT           NOT NULL,
    Company_Name   VARCHAR(150)  NOT NULL,
    Reg_Name       VARCHAR(150),
    Industry_Type  VARCHAR(100),
    PRIMARY KEY (Customer_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
        ON DELETE CASCADE
);

-- ----------------------------------------------------------
-- TABLE: Account
-- ----------------------------------------------------------
CREATE TABLE Account (
    Account_ID      INT           NOT NULL AUTO_INCREMENT,
    Account_Number  VARCHAR(30)   NOT NULL UNIQUE,
    Account_Type    ENUM('Savings','Current','Fixed_Deposit') NOT NULL,
    Balance         DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    Opening_Date    DATE          NOT NULL,
    Branch_ID       INT           NOT NULL,
    PRIMARY KEY (Account_ID),
    FOREIGN KEY (Branch_ID) REFERENCES Branch(Branch_ID)
        ON UPDATE CASCADE
);

-- ----------------------------------------------------------
-- TABLE: Savings_Account (EER Subtype)
-- ----------------------------------------------------------
CREATE TABLE Savings_Account (
    Account_ID        INT            NOT NULL,
    Min_Balance       DECIMAL(10,2)  DEFAULT 0.00,
    Interest_Rate     DECIMAL(5,2),
    Withdrawal_Limit  INT,
    PRIMARY KEY (Account_ID),
    FOREIGN KEY (Account_ID) REFERENCES Account(Account_ID)
        ON DELETE CASCADE
);

-- ----------------------------------------------------------
-- TABLE: Current_Account (EER Subtype)
-- ----------------------------------------------------------
CREATE TABLE Current_Account (
    Account_ID       INT            NOT NULL,
    Overdraft_Limit  DECIMAL(10,2)  DEFAULT 0.00,
    Monthly_Fee      DECIMAL(8,2),
    Cheque_Book      BOOLEAN        DEFAULT FALSE,
    PRIMARY KEY (Account_ID),
    FOREIGN KEY (Account_ID) REFERENCES Account(Account_ID)
        ON DELETE CASCADE
);
```

```
-- ------------------------------------------------------------
-- TABLE: Fixed_Deposit (EER Subtype)
-- ------------------------------------------------------------
CREATE TABLE Fixed_Deposit (
    Account_ID      INT             NOT NULL,
    Maturity_Date   DATE,
    Penalty_Clause  VARCHAR(255),
    PRIMARY KEY (Account_ID),
    FOREIGN KEY (Account_ID) REFERENCES Account(Account_ID)
        ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- TABLE: Customer_Account (M:N Junction)
-- ------------------------------------------------------------
CREATE TABLE Customer_Account (
    Customer_ID      INT             NOT NULL,
    Account_ID       INT             NOT NULL,
    Ownership_Share  DECIMAL(5,2)    DEFAULT 100.00,
    PRIMARY KEY (Customer_ID, Account_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Account_ID)  REFERENCES Account(Account_ID)
        ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- TABLE: Employee
-- ------------------------------------------------------------
CREATE TABLE Employee (
    Employee_ID   INT           NOT NULL AUTO_INCREMENT,
    First_Name    VARCHAR(100)  NOT NULL,
    Last_Name     VARCHAR(100)  NOT NULL,
    Position      VARCHAR(100),
    Branch_ID     INT           NOT NULL,
    PRIMARY KEY (Employee_ID),
    FOREIGN KEY (Branch_ID) REFERENCES Branch(Branch_ID)
        ON UPDATE CASCADE
);

-- ------------------------------------------------------------
-- TABLE: Manager (EER Subtype of Employee)
-- ------------------------------------------------------------
CREATE TABLE Manager (
    Employee_ID     INT           NOT NULL,
    Department      VARCHAR(100),
    Mgr_Start_Date  DATE,
    Mgr_Level       VARCHAR(50),
    PRIMARY KEY (Employee_ID),
    FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID)
        ON DELETE CASCADE
);
```

```
-- ------------------------------------------------------------
-- TABLE: Teller (EER Subtype of Employee)
-- ------------------------------------------------------------
CREATE TABLE Teller (
    Employee_ID  INT          NOT NULL,
    Counter_No   INT,
    Shift_Type   VARCHAR(50),
    PRIMARY KEY (Employee_ID),
    FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID)
        ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- TABLE: Loan_Officer (EER Subtype of Employee)
-- ------------------------------------------------------------
CREATE TABLE Loan_Officer (
    Employee_ID      INT            NOT NULL,
    Loan_Limit       DECIMAL(15,2),
    Specialization   VARCHAR(100),
    PRIMARY KEY (Employee_ID),
    FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID)
        ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- TABLE: External_Customer
-- ------------------------------------------------------------
CREATE TABLE External_Customer (
    ECustomer_ID    INT           NOT NULL AUTO_INCREMENT,
    First_Name      VARCHAR(100),
    Last_Name       VARCHAR(100),
    Bank_Name       VARCHAR(150),
    IBAN_Number     VARCHAR(50),
    Account_Number  VARCHAR(50),
    PRIMARY KEY (ECustomer_ID)
);

-- ------------------------------------------------------------
-- TABLE: Card
-- ------------------------------------------------------------
CREATE TABLE Card (
    Card_ID          INT           NOT NULL AUTO_INCREMENT,
    Card_Number      VARCHAR(20)   NOT NULL UNIQUE,
    Card_Type        ENUM('Debit','Credit') NOT NULL,
    Expiration_Date  DATE          NOT NULL,
    CVV_Code         CHAR(4)       NOT NULL,
    Issue_Date       DATE          NOT NULL,
    Card_Status      ENUM('Active','Blocked','Expired') DEFAULT 'Active',
    Account_ID       INT           NOT NULL,
    Customer_ID      INT           NOT NULL,
    PRIMARY KEY (Card_ID),
    FOREIGN KEY (Account_ID)  REFERENCES Account(Account_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
```

```
);

-- ----------------------------------------------------------
-- TABLE: Transaction
-- ----------------------------------------------------------
CREATE TABLE Transaction (
    Transaction_ID     INT            NOT NULL AUTO_INCREMENT,
    Transaction_Date   DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Transaction_Type   ENUM('Deposit','Withdrawal','Transfer') NOT NULL,
    Amount             DECIMAL(15,2)  NOT NULL CHECK (Amount > 0),
    Description        VARCHAR(255),
    Sender_Account_ID  INT,
    Receiver_Account_ID INT,
    ECustomer_ID       INT            DEFAULT NULL,
    Card_ID            INT            DEFAULT NULL,
    Employee_ID        INT            DEFAULT NULL,
    PRIMARY KEY (Transaction_ID),
    FOREIGN KEY (Sender_Account_ID)   REFERENCES Account(Account_ID),
    FOREIGN KEY (Receiver_Account_ID) REFERENCES Account(Account_ID),
    FOREIGN KEY (ECustomer_ID)        REFERENCES
External_Customer(ECustomer_ID),
    FOREIGN KEY (Card_ID)             REFERENCES Card(Card_ID),
    FOREIGN KEY (Employee_ID)         REFERENCES Employee(Employee_ID)
);

-- ----------------------------------------------------------
-- TABLE: Deposit (EER Subtype of Transaction)
-- ----------------------------------------------------------
CREATE TABLE Deposit (
    Transaction_ID  INT           NOT NULL,
    Deposit_Method  VARCHAR(100),
    Source_Account  VARCHAR(50),
    PRIMARY KEY (Transaction_ID),
    FOREIGN KEY (Transaction_ID) REFERENCES Transaction(Transaction_ID)
        ON DELETE CASCADE
);

-- ----------------------------------------------------------
-- TABLE: Withdrawal (EER Subtype of Transaction)
-- ----------------------------------------------------------
CREATE TABLE Withdrawal (
    Transaction_ID     INT           NOT NULL,
    Withdrawal_Method  VARCHAR(100),
    ATM_ID             VARCHAR(50),
    PRIMARY KEY (Transaction_ID),
    FOREIGN KEY (Transaction_ID) REFERENCES Transaction(Transaction_ID)
        ON DELETE CASCADE
);
```

```
-- ------------------------------------------------------------
-- TABLE: Transfer (EER Subtype of Transaction)
-- ------------------------------------------------------------
CREATE TABLE Transfer (
    Transaction_ID    INT            NOT NULL,
    Transfer_Fee      DECIMAL(8,2)   DEFAULT 0.00,
    Destination_Bank  VARCHAR(150),
    PRIMARY KEY (Transaction_ID),
    FOREIGN KEY (Transaction_ID) REFERENCES Transaction(Transaction_ID)
        ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- TABLE: Loan
-- ------------------------------------------------------------
CREATE TABLE Loan (
    Loan_ID           INT            NOT NULL AUTO_INCREMENT,
    Loan_Amount       DECIMAL(15,2)  NOT NULL,
    Interest_Rate     DECIMAL(5,2)   NOT NULL,
    Term              INT            NOT NULL COMMENT 'Duration in
months',
    Application_Date  DATE           NOT NULL,
    Approval_Status   ENUM('Approved','Pending','Rejected') DEFAULT
'Pending',
    Customer_ID       INT            NOT NULL,
    PRIMARY KEY (Loan_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);

-- ------------------------------------------------------------
-- TABLE: Loan_Payment
-- ------------------------------------------------------------
CREATE TABLE Loan_Payment (
    Payment_ID           INT            NOT NULL AUTO_INCREMENT,
    Loan_ID              INT            NOT NULL,
    Customer_ID          INT            NOT NULL,
    Account_ID           INT            NOT NULL,
    Payment_Date         DATE           NOT NULL,
    Payment_Amount       DECIMAL(15,2)  NOT NULL,
    Remaining_Balance    DECIMAL(15,2),
    Payment_Status       ENUM('Paid','Overdue','Pending') DEFAULT
'Pending',
    Late_Fee             DECIMAL(8,2)   DEFAULT NULL,
    Account_Cut_Off_Date DATE,
    PRIMARY KEY (Payment_ID),
    FOREIGN KEY (Loan_ID)     REFERENCES Loan(Loan_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Account_ID)  REFERENCES Account(Account_ID)
);
```

8. SQL Queries (DML)

8.1 INSERT — Sample Data

```
-- Insert Branches
INSERT INTO Branch (Branch_Code, Address, Phone) VALUES
    ('KHI-001', '123 M.A. Jinnah Road, Karachi', '021-111222333'),
    ('LHR-001', '45 Mall Road, Lahore',          '042-333444555'),
    ('ISB-001', '7 Blue Area, Islamabad',        '051-555666777');

-- Insert Customers
INSERT INTO Customer (First_Name, Last_Name, Nationality, Occupation,
Income_Level,
                      Date_of_Birth, Address, Phone, Email) VALUES
    ('Ahmed',  'Kkhan',   'Pakistani', 'Engineer',    'High',   '1990-05-
12', 'Block 4 Karachi', '0300-1234567', 'ahmed@mail.com'),
    ('Sara',   'Ali',    'Pakistani', 'Teacher',     'Medium', '1985-08-
22', 'Gulberg Lahore',  '0311-2345678', 'sara@mail.com'),
    ('Omar',   'Sheikh', 'Pakistani', 'Businessman', 'High',   '1978-01-
30', 'F-8 Islamabad',   '0321-3456789', 'omar@mail.com');

-- Insert Individual Customers
INSERT INTO Individual_Customer (Customer_ID, Nationality, Occupation,
Income_Level) VALUES
    (1, 'Pakistani', 'Engineer',    'High'),
    (2, 'Pakistani', 'Teacher',     'Medium');

-- Insert Corporate Customer
INSERT INTO Corporate_Customer (Customer_ID, Company_Name, Reg_Name,
Industry_Type) VALUES
    (3, 'Sheikh Enterprises Ltd.', 'SEL', 'Retail');

-- Insert Accounts
INSERT INTO Account (Account_Number, Account_Type, Balance, Opening_Date,
Branch_ID) VALUES
    ('ACC-10001', 'Savings',       150000.00, '2020-03-15', 1),
    ('ACC-10002', 'Current',        80000.00, '2021-07-01', 2),
    ('ACC-10003', 'Fixed_Deposit', 500000.00, '2022-01-10', 3);

-- Insert Savings subtype
INSERT INTO Savings_Account (Account_ID, Min_Balance, Interest_Rate,
Withdrawal_Limit)
    VALUES (1, 5000.00, 6.50, 5);

-- Customer-Account links
INSERT INTO Customer_Account (Customer_ID, Account_ID, Ownership_Share)
VALUES
    (1, 1, 100.00),
    (2, 2, 100.00),
    (3, 3, 100.00);

-- Insert Employees
```

```
INSERT INTO Employee (First_Name, Last_Name, Position, Branch_ID) VALUES
    ('Bilal',   'Raza',  'Manager',      1),
    ('Hina',    'Noor',  'Teller',        1),
    ('Farhan',  'Waqar', 'Loan Officer',  2);

-- Manager subtype
INSERT INTO Manager (Employee_ID, Department, Mgr_Start_Date, Mgr_Level)
    VALUES (1, 'Operations', '2019-06-01', 'Senior');

-- Teller subtype
INSERT INTO Teller (Employee_ID, Counter_No, Shift_Type)
    VALUES (2, 3, 'Morning');

-- Loan Officer subtype
INSERT INTO Loan_Officer (Employee_ID, Loan_Limit, Specialization)
    VALUES (3, 5000000.00, 'Personal Loans');

-- Insert Cards
INSERT INTO Card (Card_Number, Card_Type, Expiration_Date, CVV_Code,
                  Issue_Date, Card_Status, Account_ID, Customer_ID)
VALUES
    ('4111111111111111', 'Debit',  '2027-12-31', '123', '2022-01-10',
'Active', 1, 1),
    ('5500005555555559', 'Credit', '2026-06-30', '456', '2021-06-15',
'Active', 2, 2);

-- Insert External Customer
INSERT INTO External_Customer (First_Name, Last_Name, Bank_Name,
IBAN_Number, Account_Number)
    VALUES ('James', 'Brown', 'HSBC UK', 'GB29NWBK60161331926819', 'UK-
88899001');

-- Insert Transactions
INSERT INTO Transaction (Transaction_Type, Amount, Description,
                         Sender_Account_ID, Receiver_Account_ID,
Employee_ID) VALUES
    ('Deposit',    50000.00, 'Salary credit',        NULL, 1, 2),
    ('Withdrawal', 10000.00, 'ATM withdrawal',       1,    NULL, NULL),
    ('Transfer',   25000.00, 'Wire transfer to UK',  1,    NULL, NULL);

INSERT INTO Deposit    (Transaction_ID, Deposit_Method, Source_Account)
VALUES (1, 'Bank Transfer', 'EXT-001');
INSERT INTO Withdrawal (Transaction_ID, Withdrawal_Method, ATM_ID)
VALUES (2, 'ATM',           'ATM-KHI-007');
INSERT INTO Transfer   (Transaction_ID, Transfer_Fee, Destination_Bank)
VALUES (3, 500.00,          'HSBC UK');



-- Insert Loans
INSERT INTO Loan (Loan_Amount, Interest_Rate, Term, Application_Date,
Approval_Status, Customer_ID)
    VALUES (500000.00, 12.50, 36, '2023-03-01', 'Approved', 1);
```

```
-- Insert Loan Payments
INSERT INTO Loan_Payment (Loan_ID, Customer_ID, Account_ID, Payment_Date,
                          Payment_Amount, Remaining_Balance,
Payment_Status) VALUES
    (1, 1, 1, '2023-04-01', 18500.00, 481500.00, 'Paid'),
    (1, 1, 1, '2023-05-01', 18500.00, 463000.00, 'Paid'),
    (1, 1, 1, '2023-06-01', 18500.00, 444500.00, 'Pending');
```

8.2 UPDATE Queries

```
-- Update customer phone number
UPDATE Customer
SET    Phone = '0300-9999999'
WHERE  Customer_ID = 1;

-- Block a card
UPDATE Card
SET    Card_Status = 'Blocked'
WHERE  Card_Number = '5500005555555559';

-- Mark loan payment as overdue with late fee
UPDATE Loan_Payment
SET    Payment_Status = 'Overdue',
       Late_Fee = 500.00
WHERE  Payment_ID = 3;
```

8.3 DELETE Queries

```
-- Remove an expired card
DELETE FROM Card
WHERE  Card_Status = 'Expired';

-- Remove a rejected loan application
DELETE FROM Loan
WHERE  Approval_Status = 'Rejected';
```

8.4 SELECT Queries

Query 1 — JOIN: Customer Full Names with Their Account Numbers

```
SELECT  c.Customer_ID,
        CONCAT(c.First_Name, ' ', c.Last_Name) AS Full_Name,
        a.Account_Number,
        a.Account_Type,
        a.Balance
FROM    Customer c
JOIN    Customer_Account ca ON c.Customer_ID = ca.Customer_ID
JOIN    Account a           ON ca.Account_ID = a.Account_ID
ORDER BY c.Customer_ID;
```

Query 2 — JOIN: All Transactions with Employee and Branch

```
SELECT  t.Transaction_ID,
        t.Transaction_Date,
        t.Transaction_Type,
        t.Amount,
        CONCAT(e.First_Name, ' ', e.Last_Name) AS Processed_By,
        b.Branch_Code
FROM    Transaction t
LEFT JOIN Employee e ON t.Employee_ID = e.Employee_ID
LEFT JOIN Branch   b ON e.Branch_ID   = b.Branch_ID
ORDER BY t.Transaction_Date DESC;
```

Query 3 — GROUP BY + COUNT: Number of Accounts Per Branch

```
SELECT  b.Branch_Code,
        b.Address,
        COUNT(a.Account_ID) AS Total_Accounts
FROM    Branch b
LEFT JOIN Account a ON b.Branch_ID = a.Branch_ID
GROUP BY b.Branch_ID, b.Branch_Code, b.Address
ORDER BY Total_Accounts DESC;
```

Query 4 — Aggregation SUM: Total Loan Amount Per Customer

```
SELECT  c.Customer_ID,
        CONCAT(c.First_Name, ' ', c.Last_Name) AS Customer_Name,
        COUNT(l.Loan_ID)          AS Total_Loans,
        SUM(l.Loan_Amount)        AS Total_Borrowed,
        AVG(l.Interest_Rate)      AS Avg_Interest_Rate
FROM    Customer c
JOIN    Loan l ON c.Customer_ID = l.Customer_ID
WHERE   l.Approval_Status = 'Approved'
GROUP BY c.Customer_ID
ORDER BY Total_Borrowed DESC;
```

Query 5 — Multi-Table JOIN: Loan Repayment Summary with Account

```
SELECT  lp.Payment_ID,
        CONCAT(c.First_Name, ' ', c.Last_Name) AS Customer_Name,
        l.Loan_Amount,
        lp.Payment_Date,
        lp.Payment_Amount,
        lp.Remaining_Balance,
        lp.Payment_Status,
        a.Account_Number AS Paid_From_Account
FROM    Loan_Payment lp
JOIN    Customer c ON lp.Customer_ID = c.Customer_ID
JOIN    Loan     l ON lp.Loan_ID     = l.Loan_ID
JOIN    Account  a ON lp.Account_ID  = a.Account_ID
ORDER BY lp.Payment_Date;
```

Query 6 — Total Deposits and Withdrawals Per Account

```
SELECT  a.Account_Number,
        SUM(CASE WHEN t.Transaction_Type = 'Deposit'
                 THEN t.Amount ELSE 0 END) AS Total_Deposits,
        SUM(CASE WHEN t.Transaction_Type = 'Withdrawal'
                 THEN t.Amount ELSE 0 END) AS Total_Withdrawals,
        COUNT(t.Transaction_ID)            AS Total_Transactions
FROM    Account a
LEFT JOIN Transaction t
       ON a.Account_ID = t.Sender_Account_ID
       OR a.Account_ID = t.Receiver_Account_ID
GROUP BY a.Account_ID, a.Account_Number
ORDER BY Total_Transactions DESC;
```

8. Team Participation

The project was completed collaboratively. Each team member contributed to distinct components of the DBMS design and development process, ensuring equitable distribution of intellectual effort as required by Criterion 5 of the grading rubric.

No.	Name	Contribution
S1	HUZAIFA JAWED	ERD and EERD design, entity identification, business rules documentation
S2	FOZAN AHMAD KHAN	Functional dependency analysis and normalization (1NF to BCNF)
S3	SAIM UZ ZAMAN	Physical schema design, DDL script, and constraint enforcement
S4	M.FOUZAN ABDUL AZIZ_____	SQL query development, data analysis, joins and aggregations

9. Tools and Resources Used

Tool / Resource	Purpose	Version
MySQL	Primary DBMS for SQL schema implementation and query execution	8.0+
MySQL Workbench	GUI for designing, running, and managing the database	8.0
Microsoft Word / LibreOffice	Report documentation and formatting	-

10. Conclusion

This report presents a complete, end-to-end design and development of a Banking Database Management System. The solution progresses from a clear organizational problem analysis through conceptual (ERD), enhanced conceptual (EERD), and logical modelling, culminating in a fully normalized physical schema in Boyce-Codd Normal Form (BCNF).

The design successfully addresses all identified business requirements: multi-type accounts, joint ownership, dual customer categories, employee role specialization, full transaction lifecycle, card management, and inter-bank transfer support via External Customers. All 21 relations are free of update, insertion, and deletion anomalies.

End of Report — CT-261 Banking DBMS | Spring 2026